



Εγχειρίδιο

Contents

Επισκόπηση.....	5
Τι καλύπτει αυτό το manual	5
Κύριες δυνατότητες	5
Ορολογία και βασικές έννοιες.....	5
Τι να περιμένετε από τα επόμενα κεφάλαια	6
Απαιτήσεις συστήματος.....	7
Υποστηριζόμενες πλατφόρμες	7
Εξαρτήσεις	7
Υποχρεωτικές (CLI και GUI).....	7
Προαιρετικές (μόνο για GUI)	7
Εγκατάσταση και build στα Windows.....	8
Προαπαιτούμενα	8
Κονσόλα (CLI) – Debug.....	8
Κονσόλα (CLI) – Release.....	8
Γραφικό περιβάλλον (GUI) – Debug	8
Μοτίβο A: CMAKE_PREFIX_PATH	9
Μοτίβο B: CMAKE_PROJECT_INCLUDE (GUI include).....	9
Γραφικό περιβάλλον (GUI) – Release	9
Συχνά σφάλματα και διορθώσεις (MSVC/Qt)	9
Qt kit mismatch (MSVC vs MinGW)	9
Qt δεν εντοπίζεται από το CMake	9
Σφάλματα τύπου ‘identifier not found’ για Qt classes	10
Το GUI ανοίγει αλλά λείπουν Qt DLLs	10
Γενική σύσταση όταν αλλάζετε ρυθμίσεις build	10
Εγκατάσταση και build στο Linux	11
Προαπαιτούμενα	11
Κονσόλα (CLI) – Debug.....	11
Κονσόλα (CLI) – Release.....	11
Γραφικό περιβάλλον (GUI)	12
Configure (GUI enabled)	12

Συχνά σφάλματα και διορθώσεις (toolchain/Qt).....	12
CMake δεν βρίσκει Qt6.....	12
Σφάλματα link ή missing shared libraries κατά την εκτέλεση	12
Αλλαγές compiler	12
Χρήση CLI	13
Βασική σύνταξη	13
Επιλογή μεθόδου και προβλήματος.....	13
CSV και convergence (csv_enable/csv_convergence)	13
Run summary και σύνδεση με ρυθμίσεις.....	13
Ρυθμίσεις και αρχεία παραμετροποίησης	14
optimsolution.cfg	14
[global] και runtime overrides	15
Snapshot προσωρινού cfg ανά run.....	15
Διαχωρισμός ρυθμίσεων CLI vs GUI	15
Ελάχιστο παράδειγμα για πειράματα	16
Χρήση GUI.....	17
Δομή παραθύρου και ροή εργασίας	17
Επιλογή Method/Problem	17
Διαχείριση διάστασης (fixed vs variable)	17
Run/Stop και χειρισμός διεργασίας	17
Log.....	17
Tabs εξόδου	18
Convergence plot από CSV	18
Άξονες και επιγραφές	18
Legend και overlay	18
Export PNG.....	18
Χρωματιστό RUN SUMMARY styling.....	18
Παραδείγματα	19
Παράδειγμα εκτέλεσης CLI	19
Παράδειγμα εκτέλεσης GUI.....	20
Ελάχιστο αρχείο ρυθμίσεων (optimsolution.cfg)	21
Αντιμετώπιση προβλημάτων	22

Το optimsolution.cfg δεν εντοπίζεται από υποφακέλους.....	22
Σφάλματα build/σύνδεσης Qt (Windows).....	22
QPlainTextEdit: 'identifier not found'	22
QTextEdit: setMaximumBlockCount δεν υπάρχει	22
Mismatch Qt kit (MSVC vs MinGW)	22
Σφάλματα build/σύνδεσης Qt (Linux).....	23
Ελλιπές ή λανθασμένο CSV / convergence output.....	23
Το Run/Stop δεν τερματίζει σωστά την εκτέλεση	23
Γενική στρατηγική διάγνωσης	23
Τέλος ελέγξτε outputs (CSV, logs) και επιβεβαιώστε ότι το GUI/CLI δείχνουν στον ίδιο output_root. 23	
Βοήθεια: Γιατί 30 ανεξάρτητες εκτελέσεις;.....	24
Αξιοπιστία (επαναληψιμότητα) στη στοχαστική βελτιστοποίηση.....	24
Εγκυρότητα (ακρίβεια ως προς τον στόχο)	24
Γιατί οι «30 εκτελέσεις» είναι ένα πρακτικό πρότυπο.....	24
Συνιστώμενη πρακτική στο optimsolution	25

Επισκόπηση

Το `optimsolution` είναι ένα C++ framework βελτιστοποίησης για επαναλήψιμα, υψηλού throughput πειράματα με πληθυσμιακές μεταερευτικές μεθόδους και αριθμητικούς optimizers πάνω σε benchmark και εφαρμοστικά προβλήματα αντικειμενικής συνάρτησης. Ο βασικός στόχος του είναι η τυποποίηση της εκτέλεσης (initialization, stopping rules, logging) ώστε τα αποτελέσματα να είναι συγκρίσιμα και πλήρως αναπαραγώγιμα.

Το έργο παρέχει δύο τρόπους χρήσης: περιβάλλον κονσόλας (CLI) για batch εκτελέσεις και αυτοματοποίηση, και γραφικό περιβάλλον (GUI) για διαδραστική επιλογή μεθόδου/προβλήματος, παρακολούθηση log και οπτικοποίηση σύγκλισης μέσω CSV.

Τι καλύπτει αυτό το manual

Το παρόν manual περιγράφει βήμα-βήμα την εγκατάσταση και το build σε Windows και Linux (Debug και Release), τη χρήση του CLI και του GUI, τη λογική των ρυθμίσεων (`optimsolution.cfg`, ενότητα [global], runtime overrides), καθώς και την παραγωγή/ανάγνωση αρχείων εξόδου (run summaries, CSV, convergence).

Κύριες δυνατότητες

- Ενιαία διεπαφή εκτέλεσης μεθόδων και προβλημάτων, με τυποποιημένο logging και τελικό run summary.
- Εκτελέσεις με σαφείς προϋπολογισμούς τερματισμού (function evaluations και/ή iterations), κατάλληλες για benchmarking.
- Αναπαραγωγικότητα μέσω ανεξάρτητων runs και σταθερής αρχικοποίησης/κανόνων τερματισμού.
- Παραμετροποίηση μέσω `optimsolution.cfg`, με κοινές τιμές στο [global] και overrides ανά εκτέλεση.
- CSV export και καταγραφή convergence, ώστε να είναι δυνατή η οπτικοποίηση και η σύγκριση καμπυλών σύγκλισης.
- GUI με ξεχωριστά dropdowns για method και problem, Run/Stop έλεγχο, log σε μονογραμμικές εγγραφές και output tabs ανά run.

Ορολογία και βασικές έννοιες

Στο manual χρησιμοποιούνται οι παρακάτω όροι με σταθερή σημασία. Η συνέπεια στην ορολογία είναι κρίσιμη για την ορθή κατανόηση των ρυθμίσεων και των αρχείων εξόδου.

- Method: Ο αλγόριθμος/optimizer που εκτελείται (π.χ. DE παραλλαγές, CMA-ES, κ.λπ.).
- Problem: Η αντικειμενική συνάρτηση/benchmark με καθορισμένα bounds και dimension.
- Run: Μία ανεξάρτητη εκτέλεση ενός method πάνω σε ένα problem, υπό συγκεκριμένο budget και ρυθμίσεις.
- Budget: Το όριο υπολογισμού (evaluations, iterations) που ελέγχει πότε τερματίζει μια εκτέλεση.
- CSV / Convergence: Αρχεία εξόδου που καταγράφουν την εξέλιξη του best-so-far κατά την πορεία του run.

- Run Summary: Η τυποποιημένη περίληψη τέλους εκτέλεσης (π.χ. best/mean, timing, επιτυχία όπου ορίζεται).

Τι να περιμένετε από τα επόμενα κεφάλαια

Στα επόμενα κεφάλαια δίνονται συγκεκριμένες εντολές build, πρακτικές οδηγίες για την οργάνωση ρυθμίσεων σε CLI και GUI περιβάλλον, και παραδείγματα εκτέλεσης. Οι οδηγίες γράφονται με στόχο να μπορείτε να κάνετε clean build και να αναπαράγετε εκτελέσεις από οποιονδήποτε φάκελο εργασίας χωρίς ασάφειες.

Απαιτήσεις συστήματος

Το `optimisation` είναι project C++ που γίνεται build με CMake. Διατίθεται ως περιβάλλον κονσόλας (CLI) και προαιρετικά ως γραφικό περιβάλλον (GUI) βασισμένο σε Qt6. Οι απαιτήσεις διαφέρουν ανάλογα με το αν θα χτίσετε μόνο CLI ή και GUI.

Υποστηριζόμενες πλατφόρμες

- Windows 10/11 (x64), με Visual Studio / MSVC toolchain.
- Linux (x64), με GCC ή Clang toolchain.

Ο κώδικας είναι σχεδιασμένος για 64-bit περιβάλλον. Η ακριβής συμπεριφορά και τα απαιτούμενα packages εξαρτώνται από τη διανομή Linux και το toolchain που χρησιμοποιείτε.

Εξαρτήσεις

Υποχρεωτικές (CLI και GUI)

- CMake.
- C++ compiler με υποστήριξη C++17.

Προαιρετικές (μόνο για GUI)

- Qt6 (Core, Widgets και τα modules που απαιτεί το συγκεκριμένο GUI target).
- Κατάλληλο Qt kit που να ταιριάζει με τον compiler (π.χ. MSVC kit σε Windows).

Σε Windows, η πιο συχνή αιτία αποτυχίας είναι mismatch ανάμεσα σε compiler και Qt build (π.χ. MSVC vs MinGW). Σε Linux, το πιο συχνό είναι λανθασμένο ή μη ορατό Qt prefix path.

Εγκατάσταση και build στα Windows

Διαθέσιμο Windows installer (η πιο γρήγορη επιλογή): κατεβάστε και τρέξτε το Optimsolution.msi από:
<http://dit.uoi.gr/files/Optimsolution.msi>

Το κεφάλαιο αυτό περιγράφει τη ροή εγκατάστασης/build σε Windows 10/11 με MSVC και CMake. Οι οδηγίες είναι οργανωμένες σε CLI και GUI, σε Debug και Release. Το GUI απαιτεί Qt6 και σωστό Qt prefix path.

Προαπαιτούμενα

- Visual Studio 2022 με εγκατεστημένο Desktop development with C++.
- CMake διαθέσιμο στο PATH (ή μέσω Visual Studio).
- Qt6 MSVC kit (π.χ. C:\Qt\6.10.1\msvc2022_64) για build του GUI.

Συνιστάται οι εντολές να εκτελούνται από “Developer PowerShell for VS 2022”, ώστε να είναι διαθέσιμα τα MSVC εργαλεία.

Κονσόλα (CLI) – Debug

Clean configure/build (out-of-source):

```
Remove-Item -Recurse -Force .\build -ErrorAction SilentlyContinue
cmake -S . -B build -G "Visual Studio 17 2022" -A x64 `
  "-DCMAKE_PROJECT_INCLUDE:FILEPATH=$PWD/cmake/optimsolution_gui.cmake" `
  "-DCMAKE_PREFIX_PATH=C:\Qt\6.10.1\msvc2022_64"
cmake --build build --config Debug --target optimsolution
.\build\Debug\optimsolution.exe --help
```

Σε multi-config generator (Visual Studio) η επιλογή Debug/Release γίνεται στο build step με --config. Η παράμετρος CMAKE_BUILD_TYPE δεν χρησιμοποιείται σε αυτό το σενάριο.

Κονσόλα (CLI) – Release

```
cmake --build build --config Release --target optimsolution
.\build\Release\optimsolution.exe --help
```

Μπορείτε να χρησιμοποιήσετε το ίδιο build directory για Debug και Release. Αν αλλάξετε generator/toolchain, κάντε clean build.

Γραφικό περιβάλλον (GUI) – Debug

Το GUI target ενεργοποιείται μέσω Qt6. Υπάρχουν δύο τυπικά μοτίβα ενσωμάτωσης: α) δήλωση CMAKE_PREFIX_PATH στο configure, ή β) χρήση CMAKE_PROJECT_INCLUDE για να φορτωθεί το cmake/optimsolution_gui.cmake.

Μοτίβο A: CMAKE_PREFIX_PATH

```
Remove-Item -Recurse -Force .\build -ErrorAction SilentlyContinue
cmake -S . -B build -G "Visual Studio 17 2022" -A x64 `
  "-DCMAKE_PROJECT_INCLUDE:FILEPATH=$PWD/cmake/optimsolution_gui.cmake" `
  "-DCMAKE_PREFIX_PATH=C:\Qt\6.10.1\msvc2022_64"
cmake --build build --config Debug --target optimsolution_gui
& "C:\Qt\6.10.1\msvc2022_64\bin\windeployqt.exe" `
  --no-translations --compiler-runtime `
  ".\build\Debug\optimsolution_gui.exe"

.\build\Debug\optimsolution_gui.exe
```

Μοτίβο B: CMAKE_PROJECT_INCLUDE (GUI include)

Αν το repo έχει σχεδιαστεί ώστε το GUI να ενεργοποιείται μέσω project include, μπορείτε να περάσετε το αρχείο cmake/optimsolution_gui.cmake στο CMake. Αυτό επιτρέπει να διατηρείτε το GUI integration σε ξεχωριστό CMake include.

```
Remove-Item -Recurse -Force .\build -ErrorAction SilentlyContinue
cmake -S . -B build -G "Visual Studio 17 2022" -A x64 `
  "-DCMAKE_PROJECT_INCLUDE:FILEPATH=$PWD/cmake/optimsolution_gui.cmake" `
  "-DCMAKE_PREFIX_PATH=C:\Qt\6.10.1\msvc2022_64"
cmake --build build --config Debug --target optimsolution_gui
.\build\Debug\optimsolution_gui.exe
```

Γραφικό περιβάλλον (GUI) – Release

```
cmake --build build --config Release --target optimsolution_gui
.\build\Release\optimsolution_gui.exe
```

Συχνά σφάλματα και διορθώσεις (MSVC/Qt)

Qt kit mismatch (MSVC vs MinGW)

Βεβαιωθείτε ότι το Qt που δείχνετε στο CMake είναι χτισμένο για MSVC (π.χ. msvc2022_64). Διαφορετικά θα προκύψουν σφάλματα link ή ασυμβατότητες binaries.

Qt δεν εντοπίζεται από το CMake

- Ελέγξτε ότι το -DCMAKE_PREFIX_PATH δείχνει στο σωστό Qt prefix.

- Εναλλακτικά, ρυθμίστε Qt_DIR σε path που περιέχει τα Qt6*Config.cmake.
- Κάντε clean configure όταν αλλάζετε Qt path.

Σφάλματα τύπου ‘identifier not found’ για Qt classes

Τέτοια σφάλματα συνήθως υποδηλώνουν ότι λείπει το σωστό include ή ότι γίνεται χρήση διαφορετικού widget από αυτό που υποστηρίζεται από τις κλήσεις που κάνετε.

- Αν χρησιμοποιείτε QPlainTextEdit, βεβαιωθείτε ότι γίνεται include του <QPlainTextEdit>.
- Αν χρησιμοποιείτε QTextEdit και θέλετε περιορισμό γραμμών, χρησιμοποιήστε textEdit->document()->setMaximumBlockCount(N) και όχι textEdit->setMaximumBlockCount(...), που δεν υπάρχει στο QTextEdit.

Το GUI ανοίγει αλλά λείπουν Qt DLLs

Σε Debug/Release απαιτούνται τα αντίστοιχα Qt runtime DLLs. Αν τρέχετε εκτός Qt Creator, χρησιμοποιήστε το windeployqt ή τρέξτε από περιβάλλον όπου τα Qt bin directories είναι στο PATH. Η διαχείριση deployment είναι ξεχωριστό θέμα και συνιστάται να γίνεται με οργανωμένο packaging όταν θα διανείμετε binaries.

Γενική σύσταση όταν αλλάζετε ρυθμίσεις build

- Αν αλλάξετε generator, toolchain ή Qt prefix, σβήστε το build/ και κάντε configure από την αρχή.
- Κρατήστε ξεχωριστά build directories αν θέλετε παράλληλα διαφορετικά Qt kits ή διαφορετικές ρυθμίσεις.

Εγκατάσταση και build στο Linux

Το κεφάλαιο αυτό περιγράφει τη ροή build σε Linux με CMake και GCC/Clang. Η λογική είναι single-config (CMAKE_BUILD_TYPE) και συνιστάται out-of-source build σε ξεχωριστό build directory.

Προαπαιτούμενα

Σε Debian/Ubuntu-based distributions, εγκαταστήστε τα Qt6 development packages για το GUI target:

```
sudo apt update
```

```
sudo apt install qt6-base-dev qt6-base-dev-tools qt6-tools-dev qt6-tools-dev-tools
```

- CMake διαθέσιμο στο PATH.
- GCC ή Clang με υποστήριξη C++17.
- Για GUI: Qt6 (Core/Widgets) εγκατεστημένο και ορατό στο CMake.

Οι ονομασίες packages διαφέρουν ανά διανομή. Αν χρησιμοποιείτε system packages, βεβαιωθείτε ότι έχετε τα αντίστοιχα -dev/-devel packages για Qt6 και οποιεσδήποτε άλλες εξαρτήσεις απαιτούνται.

Κονσόλα (CLI) – Debug

Σε single-config generators, το Debug/Release ορίζεται στο configure step με CMAKE_BUILD_TYPE.

```
rm -rf build
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug -
DCMAKE_PROJECT_INCLUDE=cmake/optimsolution_gui.cmake
cmake --build build --target optimsolution
./build/optimsolution --help
```

Αν το target όνομα ή η διαδρομή του executable διαφέρει στο δικό σας setup, χρησιμοποιήστε το output του build ή το directory listing του build/ για να εντοπίσετε το παραγόμενο binary.

Κονσόλα (CLI) – Release

```
rm -rf build
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release -
DCMAKE_PROJECT_INCLUDE=cmake/optimsolution_gui.cmake
cmake --build build --target optimsolution
./build/optimsolution --help
```

Για benchmarking, συνιστάται Release build. Το Debug build χρησιμοποιείται κυρίως για ανάπτυξη και debugging.

Γραφικό περιβάλλον (GUI)

Το GUI απαιτεί Qt6 development packages. Στο README, το GUI ενεργοποιείται μέσω `CMAKE_PROJECT_INCLUDE=cmake/optimsolution_gui.cmake`.

Configure (GUI enabled)

```
rm -rf build
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug -
DCMAKE_PROJECT_INCLUDE=cmake/optimsolution_gui.cmake
cmake --build build --target optimsolution_gui
./build/optimsolution_gui
```

Για αναλυτικές οδηγίες GUI (layout, tabs, log και convergence plot), δείτε το αρχείο `optimsolution_manual.pdf`. Για οπτική αναφορά του GUI, χρησιμοποιήστε το σχήμα `optimsolution_gui.png`.

Συχνά σφάλματα και διορθώσεις (toolchain/Qt)

CMake δεν βρίσκει Qt6

- Ελέγξτε ότι εγκαταστήσατε τα Qt6 development packages (π.χ. `qt6-base-dev` ή αντίστοιχα).
- Αν χρησιμοποιείτε Qt από installer, βεβαιωθείτε ότι το `CMAKE_PREFIX_PATH` δείχνει στο σωστό Qt prefix.
- Κάντε clean configure (σβήστε το build/) όταν αλλάζετε Qt paths ή toolchain.

Σφάλματα link ή missing shared libraries κατά την εκτέλεση

Αν το executable χτίζεται αλλά αποτυγχάνει να τρέξει λόγω shared libraries, ελέγξτε ότι οι runtime βιβλιοθήκες είναι διαθέσιμες (ldd) και ότι το περιβάλλον σας (`LD_LIBRARY_PATH` / `rpath`) είναι σωστά ρυθμισμένο.

Αλλαγές compiler

Αν αλλάξετε από GCC σε Clang (ή το αντίστροφο), προτιμήστε clean build σε νέο build directory ώστε να αποφύγετε μείξη artifacts.

Χρήση CLI

Το CLI του `optimsolution` είναι σχεδιασμένο για batch εκτελέσεις και αυτοματοποίηση. Στόχος είναι να μπορείτε να εκτελείτε έναν `method` πάνω σε ένα `problem` με σαφές `budget` και σταθερή συμπεριφορά τερματισμού, καταγράφοντας τυποποιημένα `logs` και `run summary`.

Βασική σύνταξη

Χρησιμοποιήστε πάντα την εντολή βοήθειας για να δείτε τη διαθέσιμη σύνταξη στο δικό σας `build`.

```
optimsolution --help
```

Επιλογή μεθόδου και προβλήματος

Σημείωση: Στο README, η επιλογή `method/problem` γίνεται με `positional arguments` (π.χ. "`optimsolution arg tersoffc`"). Χρησιμοποιήστε "`optimsolution --help`" για να επιβεβαιώσετε τυχόν προαιρετικά `arguments` που υποστηρίζει το δικό σας `build`.

Τυπική ροή: ορίζετε `method` και `problem` (με τα `short names`).

```
optimsolution <METHOD> <PROBLEM>
```

```
optimsolution <METHOD> <PROBLEM>
```

Για `fixed-dimension` προβλήματα, το CLI μπορεί να αγνοεί το `--dim` ή να έχει εναλλακτικό `mode`.

CSV και convergence (`csv_enable/csv_convergence`)

- `csv_enable`: ενεργοποιεί/απενεργοποιεί τη δημιουργία CSV.
- `csv_convergence`: ενεργοποιεί την καταγραφή `convergence` (`best-so-far`).

Το GUI χρησιμοποιεί αυτά τα CSV για `convergence plots`.

Run summary και σύνδεση με ρυθμίσεις

Στο τέλος κάθε `run` εκτυπώνεται `run summary`. Οι περισσότερες παράμετροι διαβάζονται από το `optimsolution.cfg`, με `[global]` `defaults` και `overrides` (`sections` ή `runtime overrides`).

Ρυθμίσεις και αρχεία παραμετροποίησης

Το `optimsolution` βασίζεται σε ρητή παραμετροποίηση μέσω αρχείου ρυθμίσεων, με στόχο την αναπαραγωγικότητα και την τυποποίηση εκτελέσεων. Η κύρια πηγή ρυθμίσεων είναι το `optimsolution.cfg` στο `root` του αποθετηρίου. Το `manual` υιοθετεί την αρχή ότι το `[global]` περιέχει κοινές προεπιλογές, ενώ οι εξειδικεύσεις γίνονται είτε σε επιμέρους `sections` είτε ως `runtime overrides`.

`optimsolution.cfg`

Το `optimsolution.cfg` είναι αρχείο τύπου `INI` με `sections`. Περιλαμβάνει γενικές ρυθμίσεις, ρυθμίσεις για μεθόδους και προβλήματα (όπου εφαρμόζεται), καθώς και επιλογές παραγωγής εξόδων (`logs/CSV`).

Ενδεικτική δομή (illustrative):

```
[global]
population = 200
max_iters  = 500000
max_evals  = 150000
seed_base  = 4242
runs       = 30
success_tol = 1e-8
local_rate  = 0.00
local_method = lbfgs
end_local_refine = 0
end_local_method = lbfgs ; or bfgs / nm / gd
summary_enable = 1

[stop]
rule      = maxevals ;
bss|wss|tss|boss|srs|irs|doublebox|maxevals|all|none
eps       = 1e-10

[init]
type = uniform ; uniform | normal | cauchy | laplace | lognormal
| exponential | beta | levy | lhs | halton | oppositional
```

Example method overrides (ARQ):

```
[arq]
population    = 100
pbest_frac    = 0.12
F             = 0.1
CR            = 0.9
archive_rate  = 1.5
batch_frac    = 0.5
local_rate    = 0.00
local_method  = lbfgs
```

Οι ακριβείς ονομασίες keys/sections εξαρτώνται από την υλοποίηση του project. Το παράδειγμα δείχνει τη φιλοσοφία: κοινές τιμές στο [global], ειδικές τιμές ανά method/problem όπου απαιτείται, και ξεχωριστό section για GUI state.

[global] και runtime overrides

Η ενότητα [global] λειτουργεί ως default layer. Οι επιμέρους sections (π.χ. method-specific) ή τα runtime overrides υπερισχύουν των global τιμών. Αυτό επιτρέπει να έχετε μία σταθερή βάση και να τροποποιείτε μόνο ό,τι χρειάζεται ανά run.

- Layer 1: [global] defaults.
- Layer 2: section overrides (π.χ. method/problem).
- Layer 3: runtime overrides (παράμετροι που περνιούνται στη γραμμή εντολών ή στο GUI πριν το run).

Τα runtime overrides είναι χρήσιμα για πειράματα ευαισθησίας (parameter sensitivity) ή για γρήγορες δοκιμές χωρίς να αλλάζετε μόνιμα το cfg. Κρίσιμο σημείο: τα overrides πρέπει να καταγράφονται μαζί με το run για πλήρη αναπαραγωγικότητα.

Snapshot προσωρινού cfg ανά run

Για να διασφαλιστεί ότι κάθε run είναι πλήρως τεκμηριωμένο, το optimsolution δημιουργεί ένα προσωρινό snapshot του τελικού configuration που χρησιμοποιήθηκε (global + overrides), και το αποθηκεύει μαζί με τα outputs του run. Το snapshot αυτό επιτρέπει να αναπαράγετε ακριβώς μία εκτέλεση ακόμη και αν αλλάξετε αργότερα το βασικό optimsolution.cfg.

Συνιστώμενη πρακτική:

- Αποθήκευση του snapshot cfg μέσα στο output directory του run (μαζί με CSV και summary).
- Καταγραφή στο log του πλήρους path του snapshot και των overrides που εφαρμόστηκαν.
- Χρήση μοναδικού run id ή timestamp για αποφυγή overwrites.

Διαχωρισμός ρυθμίσεων CLI vs GUI

Το CLI και το GUI μοιράζονται κοινό πυρήνα ρυθμίσεων για την εκτέλεση optimizers (budgets, CSV, output paths, κ.λπ.). Ωστόσο, το GUI χρειάζεται και επιπλέον ρυθμίσεις κατάστασης/εμπειρίας χρήστη (π.χ. τελευταία επιλογή method/problem, layout, στήλες/ορατότητα tabs). Για να αποφευχθεί σύγχυση, οι ρυθμίσεις πρέπει να διαχωρίζονται καθαρά.

- CLI settings: οτιδήποτε επηρεάζει το αποτέλεσμα μιας εκτέλεσης (π.χ. budgets, seed, method parameters, CSV settings).
- GUI settings: οτιδήποτε αφορά UI state και παρουσίαση (π.χ. last selected method/problem, window geometry, plot options).

Πρακτικά, αυτό σημαίνει ότι το optimsolution.cfg μπορεί να περιέχει ξεχωριστό section για GUI (π.χ. [gui]) ή/και να χρησιμοποιείται ξεχωριστό αρχείο ρυθμίσεων GUI. Σε κάθε περίπτωση, το run snapshot πρέπει να περιλαμβάνει μόνο τις ρυθμίσεις που επηρεάζουν την εκτέλεση και όχι καθαρά UI state, ώστε να παραμένει καθαρό το πειραματικό αποτύπωμα.

Ελάχιστο παράδειγμα για πειράματα

Για τυπικά benchmarking runs, κρατήστε τα εξής στο [global] και αλλάζετε μόνο ό,τι είναι αναγκαίο ανά method:

```
[global]
max_evals      = 200000
csv_enable     = 1
csv_convergence = 1
output_root    = outputs
```

Στη συνέχεια, ορίστε τα method-specific parameters σε section, ώστε να είναι εμφανές τι αλλάζει ανά method. Τα runtime overrides χρησιμοποιούνται μόνο όταν θέλετε ad-hoc μεταβολές και πρέπει πάντα να αποτυπώνονται στο snapshot.

Χρήση GUI

Το GUI του `optimsolution` παρέχει διαδραστική εκτέλεση μεθόδων/προβλημάτων, με έμφαση στη σαφή ροή εργασίας, στην επιθεώρηση log και στη γρήγορη οπτικοποίηση σύγκλισης μέσω CSV. Για οπτική αναφορά του layout, χρησιμοποιήστε το σχήμα `optimsolution_gui.png`. Για πλήρεις οδηγίες χρήσης, ανατρέξτε στο `optimsolution_manual.pdf`.

Δομή παραθύρου και ροή εργασίας

Η βασική ροή είναι: επιλογή Method, επιλογή Problem, ρύθμιση παραμέτρων/budgets (όπου εμφανίζονται), και εκτέλεση με Run. Κάθε εκτέλεση παράγει log και ανοίγει output tab με όνομα `method+problem`.

Επιλογή Method/Problem

Η επιλογή γίνεται από δύο ξεχωριστά dropdowns. Το dropdown των μεθόδων εμφανίζει την ονομασία ως “Full name (short)”, ώστε να είναι ταυτόχρονα αναγνώσιμη και σύντομη.

- Method dropdown: “Full name (short)”.
- Problem dropdown: αναγνωρίσιμο short/full name ανάλογα με το project.

Διαχείριση διάστασης (fixed vs variable)

Για προβλήματα σταθερής διάστασης (fixed-dimension), το GUI δεν εμφανίζει επιλογή διάστασης και δεν εφαρμόζει auto-retry σε dimension. Για προβλήματα μεταβλητής διάστασης, εμφανίζεται πεδίο/επιλογή dimension όπου απαιτείται.

- Fixed-dimension: απόκρυψη dimension control και αποφυγή auto-retry.
- Variable-dimension: εμφανίζεται dimension control και περνά στο run ως παράμετρος.

Run/Stop και χειρισμός διεργασίας

Το κουμπί εκτέλεσης λειτουργεί ως toggle Run/Stop. Κατά την εκτέλεση, το Stop τερματίζει την τρέχουσα διεργασία με ασφαλή τρόπο, ώστε να μην αφήνονται “ορφανά” processes.

- Run: εκκίνηση εκτέλεσης με το τρέχον configuration.
- Stop: τερματισμός της τρέχουσας εκτέλεσης και επιστροφή σε κατάσταση αναμονής.

Log

Το log προβάλλεται ως σειρά μονογραμμικών εγγραφών χωρίς κενές γραμμές. Αυτό διευκολύνει την ανάγνωση και την εύκολη αντιγραφή πληροφοριών (π.χ. best values ανά checkpoint) χωρίς θόρυβο μορφοποίησης.

Tabs εξόδου

Τα outputs οργανώνονται σε πολλαπλά tabs. Κάθε tab αντιστοιχεί σε ένα run (ή σε μία λογική εκτέλεση) και ονομάζεται με βάση method+problem. Τα tabs είναι κλεισίματα (closable), ώστε να καθαρίζετε γρήγορα το workspace.

Convergence plot από CSV

Το convergence plot δημιουργείται από τα CSV outputs (csv_enable/csv_convergence). Το GUI διαβάζει τα CSV και εμφανίζει καμπύλες best-so-far. Το plot είναι σχεδιασμένο ώστε να υποστηρίζει σύγκριση πολλαπλών προβλημάτων για την ίδια μέθοδο.

Άξονες και επιγραφές

- X-axis title: "Iterations".
- Y-axis title: "Best solution"

Legend και overlay

Το legend περιλαμβάνει το problem short name και τη διάσταση. Επιτρέπεται overlay πολλαπλών προβλημάτων για το ίδιο method, ώστε να συγκρίνετε καμπύλες σε ένα ενιαίο γράφημα.

Export PNG

Το plot μπορεί να εξαχθεί σε PNG με 300 DPI για χρήση σε αναφορές και άρθρα. Η εξαγωγή πρέπει να διατηρεί καθαρούς τίτλους αξόνων και ευανάγνωστο legend.

Χρωματιστό RUN SUMMARY styling

Το GUI εμφανίζει στο τέλος της εκτέλεσης ένα RUN SUMMARY με χρωματιστή μορφοποίηση, ώστε οι κρίσιμες τιμές (π.χ. best, success, timing) να είναι άμεσα ορατές. Η σύνοψη αυτή συμπληρώνει το raw log και διευκολύνει γρήγορη αξιολόγηση.

Παραδείγματα

Το κεφάλαιο αυτό παρέχει πρακτικά παραδείγματα για γρήγορη εκτέλεση από CLI και GUI, καθώς και ένα ελάχιστο παράδειγμα ρυθμίσεων. Τα παραδείγματα είναι σκόπιμα “template-style”, ώστε να προσαρμοστούν εύκολα στα πραγματικά ονόματα methods/problems του δικού σας build.

Παράδειγμα εκτέλεσης CLI

Εκτέλεση ενός run με explicit budgets και seed.

optimsolution <METHOD> <PROBLEM>

```
Windows PowerShell
[Run 30 | iter 2491 | evals 149794] best_f = -32.013207087264
[Run 30 | iter 2492 | evals 149854] best_f = -32.013207087264
[Run 30 | iter 2493 | evals 149914] best_f = -32.013207087264
[Run 30 | iter 2494 | evals 149974] best_f = -32.013207087264
[Run 30 | iter 2495 | evals 150000] best_f = -32.013207087264

===== RUN SUMMARY =====

----- GENERAL -----
Method:                arq
Method full name:      ARQ: Adaptive RTR with Quarantine
Problem:               tersoffc (Dim=24)

----- PROBLEM INFO -----
Problem full name:     TersoffC Si(C) Cluster Potential (CEC2011)
Problem modality:      multimodal
Problem separability:  non-separable
Problem category:      molecular modelling / cluster optimization
Bounds:                non-uniform per dimension

----- EXPERIMENT CONFIG -----
Population:            100
Init distribution:      uniform
Runs:                  30
Max evals/run:         150000
Max iters/run:         500000
Stop rule:             maxevals
Success criterion:     f within 3.42574055152e-08 of best-of-runs (-34.2574055152)

----- PERFORMANCE -----
Best f (min):          -34.257405515172
Best f (mean +- sd):   -32.692619515222 (+- 0.672258983381)
Best f (median, Q1-Q3): median=-32.7007 [Q1=-33.2032, Q3=-32.237]
Evals to reach target: 150000 (+- 0)

----- CALLS & SUCCESS -----
Evals (mean over runs): 150000 (+- 0)
Grad calls (mean over runs): 0 (+- 0)
Grad/Func calls ratio (mean): 0
Success rate:          3.33333% (1/30)
Best run (index,f,evals): #1 (f=-34.2574, evals=150000)
Worst run (index,f,evals): #2 (f=-31.3558, evals=150000)

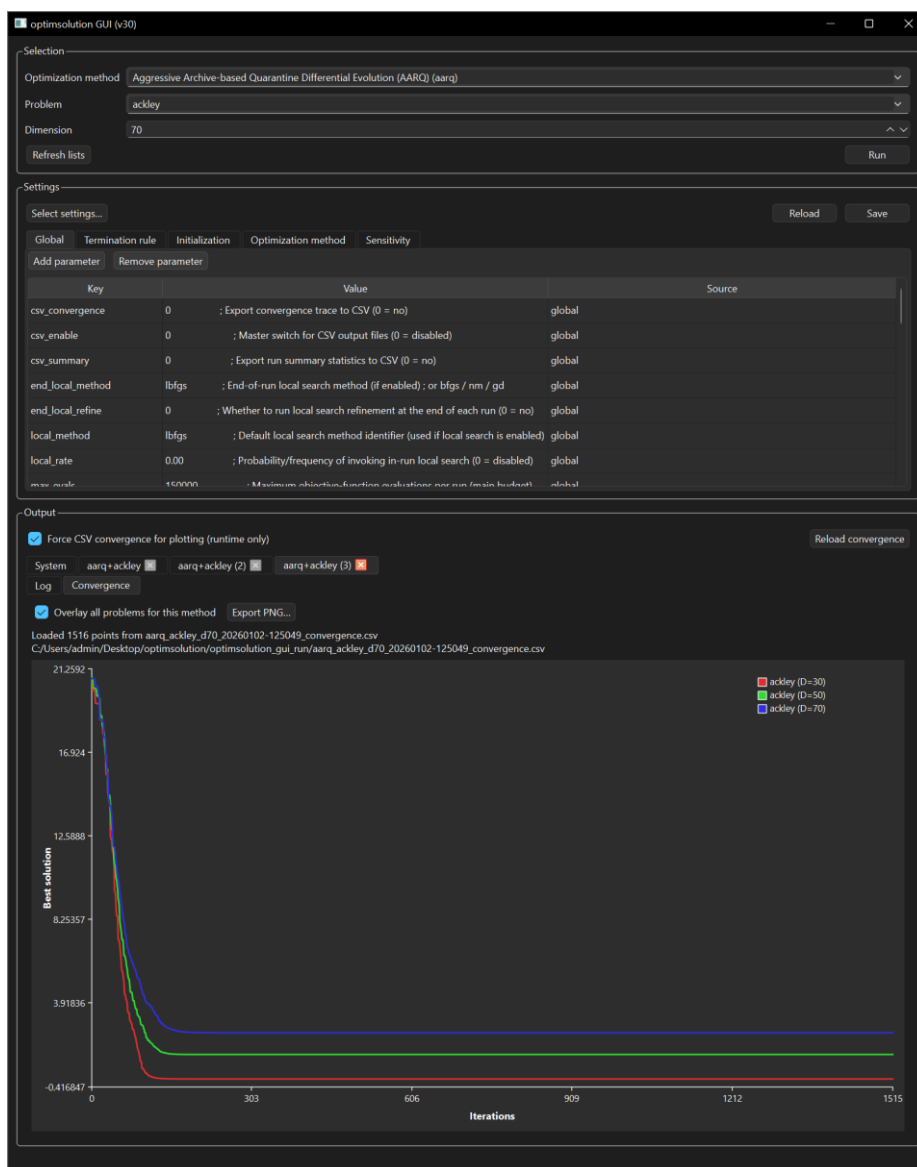
----- TIMING & MEMORY -----
Iter time (mean of means): 0.006155 s (+- 0.000175 s)
Iter time (p95 across runs): 0.007548 s (+- 0.000631 s)
Time per run (mean): 15.329133 s
Total wall time (all runs): 459.873993 s
Memory peak (mean): 5.87018 MB (+- 0.00499226 MB)

----- LOCAL SEARCH -----
Local search (in-run): none
Local search (in-end): none

----- SUMMARY HIGHLIGHTS -----
Best f (min):          -34.2574
Success rate:          3.33333% (1/30)
PS C:\Users\admin\Desktop\optimsolution\build>
```

Παράδειγμα εκτέλεσης GUI

- 1) Εκκινήστε το GUI executable. Σε Windows, αυτό είναι συνήθως στο build/Debug ή build/Release.
- 2) Επιλέξτε Method από το dropdown (μορφή “Full name (short)”).
- 3) Επιλέξτε Problem από το αντίστοιχο dropdown.
- 4) Αν το πρόβλημα είναι variable-dimension, ορίστε dimension. Αν είναι fixed-dimension, το GUI δεν θα εμφανίσει dimension control.
- 5) Πατήστε Run. Παρακολουθήστε το log (μονογραμμικές εγγραφές).
- 6) Με το τέλος του run, επιθεωρήστε το RUN SUMMARY και το output tab (method+problem).
- 7) Αν έχετε ενεργοποιήσει csv_enable/csv_convergence, εμφανίστε το convergence plot και, αν χρειάζεται, εξάγετε PNG (300 DPI).



Ελάχιστο αρχείο ρυθμίσεων (optimsolution.cfg)

Το παρακάτω είναι ένα ελάχιστο, πρακτικά χρήσιμο baseline για benchmarking. Προσαρμόστε max_evals και output_root σύμφωνα με το workflow σας.

```
[global]
population = 200
max_iters = 500000
max_evals = 150000
runs      = 30
seed_base = 4242
csv_enable = 1
csv_convergence = 1
```

```
[stop]
rule = maxevals
```

```
[init]
type = uniform
```

Αντιμετώπιση προβλημάτων

Το κεφάλαιο αυτό συγκεντρώνει συχνά προβλήματα που εμφανίζονται κατά την εγκατάσταση, το build ή την εκτέλεση του `optimsolution`, και προτείνει πρακτικές διορθώσεις. Στόχος είναι να εντοπίζονται γρήγορα τα αίτια (`working directory`, `paths`, `Qt mismatch`, `CSV outputs`) χωρίς ad-hoc αλλαγές που δυσκολεύουν την αναπαραγωγιμότητα.

Το `optimsolution.cfg` δεν εντοπίζεται από υποφάκελους

Σύμπτωμα: όταν εκτελείτε το CLI/GUI από `build/Debug`, `build/Release` ή άλλο υποφάκελο, η εφαρμογή δεν βρίσκει το `optimsolution.cfg`, παρότι υπάρχει στο `root` του `repo`.

Πρακτικές λύσεις:

- Ρυθμίστε το `working directory` του `run` στο `root` του `repo` (ιδίως όταν τρέχετε από IDE).
- Χρησιμοποιήστε λογική `discovery` που ανεβαίνει γονικούς φακέλους μέχρι να βρει `optimsolution.cfg`.
- Υποστηρίξτε `explicit` παράμετρο τύπου `--config <path_to_cfg>` για να δίνετε πλήρες `path`.
- Καταγράψτε στο `log` το `path` του `cfg` που τελικά φορτώθηκε, για να εντοπίζεται άμεσα το πρόβλημα.

Σημείωση: το `manual` θεωρεί ως επιθυμητή συμπεριφορά ότι το `cfg` στο `root` πρέπει να εντοπίζεται ανεξάρτητα από το `working directory`, μέσω `robust discovery` ή `explicit configuration path`.

Σφάλματα build/σύνδεσης Qt (Windows)

QPlainTextEdit: 'identifier not found'

Αν εμφανιστεί σφάλμα ότι το `QPlainTextEdit` δεν αναγνωρίζεται, συνήθως λείπει `include` ή δεν έχει γίνει `link` στα σωστά Qt modules.

- Προσθέστε `include: <QPlainTextEdit>`.
- Βεβαιωθείτε ότι το `target` κάνει `link` στο `Qt6::Widgets`.
- Κάντε `clean build` αν αλλάξατε modules ή Qt prefix.

QTextEdit: setMaximumBlockCount δεν υπάρχει

Το `setMaximumBlockCount` είναι API του `QPlainTextEdit`. Για `QTextEdit` πρέπει να περιορίσετε `blocks` μέσω του `document()`.

```
textEdit->document() ->setMaximumBlockCount(1000);
```

Ελέγξτε ότι έχετε `include` του `<QTextEdit>` και `<QTextDocument>` όπου απαιτείται.

Mismatch Qt kit (MSVC vs MinGW)

Χρησιμοποιήστε Qt που είναι χτισμένο για τον ίδιο compiler με τον οποίο χτίζετε το project. Σε MSVC build, το Qt prefix πρέπει να είναι msvc2022_64 (ή αντίστοιχο).

Σφάλματα build/σύνδεσης Qt (Linux)

Σε Linux, τα πιο συχνά αίτια είναι ότι λείπουν Qt6 development packages ή ότι το CMake δεν βρίσκει το Qt επειδή το prefix path δεν είναι σωστό.

- Επιβεβαιώστε εγκατάσταση Qt6 dev packages (distribution-specific).
- Αν χρησιμοποιείτε Qt installer, ορίστε σωστό `-DCMAKE_PREFIX_PATH` στο configure.
- Διαγράψτε build/ και επαναλάβετε configure όταν αλλάζετε Qt paths.

Ελλιπές ή λανθασμένο CSV / convergence output

Σύμπτωμα: το GUI δεν εμφανίζει plot ή εμφανίζει κενό/λανθασμένο γράφημα, παρότι έχει ολοκληρωθεί run.

Έλεγχοι και διορθώσεις:

- Ελέγξτε ότι `csv_enable=1` και `csv_convergence=1` στο effective configuration (snapshot).
- Ελέγξτε ότι το output directory που χρησιμοποιεί το run είναι αυτό που νομίζετε (π.χ. `output_root`).
- Επιβεβαιώστε ότι δημιουργούνται CSV αρχεία και ότι έχουν δεδομένα (όχι μόνο headers).
- Αν το plotting βασίζεται σε naming conventions, βεβαιωθείτε ότι τα filenames ταιριάζουν στο expected pattern.

Αν το CSV είναι σωστό αλλά το plot παραμένει κενό, ελέγξτε ότι το GUI διαβάζει το ίδιο output directory με αυτό που γράφει το run και ότι δεν υπάρχουν πολλαπλά working directories που οδηγούν σε διαφορετικά outputs.

Το Run/Stop δεν τερματίζει σωστά την εκτέλεση

Σύμπτωμα: πατάτε Stop αλλά η εκτέλεση συνεχίζει, ή παραμένει “κολλημένη” διεργασία.

- Βεβαιωθείτε ότι το GUI χρησιμοποιεί σωστό process handle και κάνει terminate/kill στο σωστό PID.
- Καταγράψτε στο log την αλλαγή κατάστασης (Run -> Stop) και το αποτέλεσμα τερματισμού.
- Αν υπάρχουν child processes, εξασφαλίστε ότι τερματίζονται και αυτά (ανάλογα με την υλοποίηση).

Γενική στρατηγική διάγνωσης

- Πρώτα ελέγξτε paths/working directory και το effective cfg snapshot.
- Έπειτα ελέγξτε toolchain/Qt mismatches και κάντε clean configure όταν αλλάζετε βασικές παραμέτρους.

Τέλος ελέγξτε outputs (CSV, logs) και επιβεβαιώστε ότι το GUI/CLI δείχνουν στον ίδιο `output_root`.

Βοήθεια: Γιατί 30 ανεξάρτητες εκτελέσεις;

Πολλοί βελτιστοποιητές που υλοποιούνται στο `optimsolution` είναι στοχαστικοί (π.χ., εξελικτικές και σμηνουργικές μέθοδοι). Σε τέτοιους αλγόριθμους, το παρατηρούμενο αποτέλεσμα δεν είναι μία μοναδική ντετερμινιστική τιμή· είναι μία τυχαία μεταβλητή που εξαρτάται από πηγές τυχαιότητας όπως ο αρχικός πληθυσμός, οι επιλογές μετάλλαξης/διασταύρωσης και άλλες διαδικασίες δειγματοληψίας. Για τον λόγο αυτό, μία μόνο εκτέλεση μπορεί να μην είναι αντιπροσωπευτική (είτε ασυνήθιστα καλή είτε ασυνήθιστα κακή) και συμπεράσματα που βασίζονται σε μία εκτέλεση γενικά δεν είναι στατιστικά τεκμηριώσιμα.

Αξιοπιστία (επαναληψιμότητα) στη στοχαστική βελτιστοποίηση

Στην επιστήμη των μετρήσεων, η αξιοπιστία αναφέρεται στη συνέπεια μιας διαδικασίας μέτρησης όταν επαναλαμβάνεται υπό πανομοιότυπες συνθήκες. Στη στοχαστική βελτιστοποίηση, η αξιοπιστία αξιολογείται επαναλαμβάνοντας τον αλγόριθμο με την ίδια πειραματική διαμόρφωση (ίδιο πρόβλημα, όρια, προϋπολογισμό και παραμετροποίηση), μεταβάλλοντας μόνο τον τυχαίο σπόρο (`random seed`). Αν ο επιλύτης παράγει παρόμοιες κατανομές επίδοσης στις επαναλήψεις (π.χ., σταθερό μέσο/διάμεσο και εύλογη διασπορά), τότε τα πειραματικά αποτελέσματα θεωρούνται επαναλήψιμα και η μέθοδος μπορεί να αξιολογηθεί με εμπιστοσύνη.

Εγκυρότητα (ακρίβεια ως προς τον στόχο)

Η εγκυρότητα αναφέρεται στο κατά πόσο μια μέτρηση αποτυπώνει την «αληθινή» ποσότητα ενδιαφέροντος. Στη συγκριτική αξιολόγηση (`benchmarking`) βελτιστοποίησης, η εγκυρότητα συνδέεται με το αν το πειραματικό πρωτόκολλο μπορεί αξιόπιστα να αντανakλά την ικανότητα του αλγορίθμου να προσεγγίζει τις καλύτερες επιτεύξιμες τιμές του αντικειμενικού στόχου υπό τον δεδομένο προϋπολογισμό. Ένα αξιόπιστο πρωτόκολλο αποτελεί προαπαιτούμενο για την εγκυρότητα: χωρίς επαναληψιμότητα, είναι αδύνατο να διακριθεί η πραγματική αλγοριθμική επίδοση από τυχαίες διακυμάνσεις.

Γιατί οι «30 εκτελέσεις» είναι ένα πρακτικό πρότυπο

Η χρήση πολλών ανεξάρτητων εκτελέσεων απαιτείται για να:

- εκτιμηθούν το μέτρο κεντρικής τάσης (μέσος/διάμεσος) και η διασπορά (τυπική απόκλιση / ενδοτεταρτημοριακό εύρος),
- ποσοτικοποιηθεί η πιθανότητα επιτυχίας (ρυθμός επιτυχίας) υπό σταθερό προϋπολογισμό,
- υποστηριχθούν συμπερασματικές συγκρίσεις μεταξύ μεθόδων μέσω μη παραμετρικών ελέγχων (π.χ., `Wilcoxon`, `Friedman`) που προϋποθέτουν ανεξάρτητα δείγματα.

Η επιλογή 30 ανεξάρτητων εκτελέσεων χρησιμοποιείται ευρέως ως πρακτικός συμβιβασμός μεταξύ στατιστικής σταθερότητας και υπολογιστικού κόστους. Με περίπου 30 δείγματα, τα συνοπτικά στατιστικά και οι εμπειρικοί ρυθμοί επιτυχίας τείνουν να γίνονται αισθητά πιο σταθεροί σε σύγκριση

με πολύ μικρά μεγέθη δείγματος, ενώ το μέγεθος αυτό είναι επίσης επαρκές ώστε πολλές τυπικές μη παραμετρικές συγκρίσεις να έχουν ικανοποιητική ισχύ υπό συνήθεις προϋπολογισμούς benchmarking.

Συνιστώμενη πρακτική στο optimsolution

- Χρησιμοποιήστε 30 εκτελέσεις με διαφορετικούς τυχαίους σπόρους για κάθε τετράδα (πρόβλημα, διάσταση, μέθοδος, διαμόρφωση).
- Αναφέρετε, τουλάχιστον: καλύτερο-από-τις-εκτελέσεις, μέσος $\pm$ τυπική απόκλιση, διάμεσος (Q1–Q3) και ρυθμός επιτυχίας.
- Κρατήστε όλα τα υπόλοιπα σταθερά (προϋπολογισμό, όρια, τύπο αρχικοποίησης, ρυθμίσεις παραμέτρων), ώστε η παρατηρούμενη μεταβλητότητα να οφείλεται στη στοχαστικότητα και όχι σε ανεξέλεγκτες αλλαγές.